

Você atuará como um engenheiro de software sênior responsável por analisar, projetar, implementar, testar e documentar uma refatoração crítica no sistema de Rotas.

O sistema utiliza React/TypeScript no frontend, Supabase/PostgreSQL no backend e Supabase Edge Functions para integração com o ERP e sincronização de dados.

A tarefa envolve principalmente:

- módulo Empresa/Clientes, relacionado ao arquivo `clientes.tsx`;
- módulo KPI, relacionado ao arquivo `kpi.tsx`;
- Edge Function atual `kpi-sync-daily`;
- integração com uma API do ERP;
- leitura do parâmetro `AssociadoTitular`;
- atualização dos dados no banco do sistema de Rotas;
- propagação da atualização para os dados utilizados pelo módulo KPI.

Não trate esta tarefa como um ajuste superficial.

Primeiro compreenda integralmente a implementação existente. Depois apresente um diagnóstico técnico. Em seguida proponha uma arquitetura de transição compatível com o estado atual. Por fim, implemente a solução completa, com foco em desempenho, confiabilidade, idempotência, observabilidade e capacidade de crescimento.

1. Contexto funcional

Existe uma API do ERP que é chamada individualmente para cada código de empresa ou associado.

A API utiliza verbo HTTP `GET`.

Apesar de semanticamente não ser o comportamento ideal para um `GET`, essa API existente deve ser considerada uma dependência externa que não será alterada neste momento.

Ao consultar um código, a API:

1. lê no ERP o parâmetro `AssociadoTitular`;
2. retorna os dados relacionados ao código;
3. permite que o sistema atualize o registro correspondente no banco de dados do Rotas;
4. após a atualização, os dados utilizados pelo módulo KPI precisam ser sincronizados ou recalculados.

Exemplo conceitual:

```
Código 21344
↓
Consulta à API do ERP
↓
Leitura de AssociadoTitular
↓
```

Atualização da empresa no banco do Rotas

↓

Atualização ou recálculo dos dados correspondentes no KPI

Atualmente existem aproximadamente 2.600 códigos únicos.

Esse volume deve crescer continuamente, podendo chegar a:

- 3.000 códigos;
- 5.000 códigos;
- 10.000 códigos;
- volumes ainda maiores no futuro.

A solução não pode depender de um limite fixo de códigos.

2. Objetivo principal de negócio

Ao clicar no botão “Sincronizar”, o processo completo deve ser executado rapidamente.

A meta desejada é:

3.000 códigos processados em aproximadamente 2 a 3 minutos

Essa meta inclui, sempre que aplicável:

- consulta dos códigos no ERP;
- leitura do campo `AssociadoTitular`;
- comparação com os dados atuais;
- atualização do banco;
- registro de alterações;
- propagação para o KPI;
- finalização segura do run;
- atualização do status exibido ao usuário.

O objetivo não é apenas impedir que a função trave.

O objetivo é criar um pipeline de sincronização:

- rápido;
- paralelo;
- escalável;
- rastreável;
- recuperável;
- idempotente;
- controlado;
- confiável;
- seguro contra duplicidades;
- capaz de concluir milhares de códigos em poucos minutos.

3. Estado atual conhecido

O sistema possui uma implementação existente no `kpi-sync-daily`.

A implementação atual aparentemente possui:

- ação `start`;
- ação `continue`;
- ação `stop`;
- lock chamado `kpi_daily`;
- registro de runs;
- snapshots;
- registros de erros do run;
- progresso baseado em `processed_codes`;
- progresso baseado em `remaining_codes`;
- contador de `failed_codes`;
- contador de `changed_codes`;
- status `running`;
- polling realizado pelo frontend;
- chamada direta da Edge Function pelo frontend;
- mecanismo de encadeamento de chunks;
- autochamada da própria Edge Function;
- parada manual emergencial.

O problema observado é que o run pode ficar preso no worker atual.

Exemplo de status observado:

```
{
  "status": "running",
  "total_codes": 2606,
  "processed_codes": 0,
  "remaining_codes": 2606,
  "failed_codes": 0,
  "changed_codes": 0,
  "current_code": "21344",
  "current_stage": "processing_batch"
}
```

O banner permanece mostrando que um código está sendo processado, porém o pipeline não avança para o próximo grupo.

Isso indica que a arquitetura atual ainda depende excessivamente de:

- uma execução longa;
- uma cadeia frágil de autochamadas;
- estado mantido em memória;
- promessa assíncrona que pode morrer junto com a função;
- continuidade não garantida após timeout, encerramento ou falha transitória.

4. Restrição principal da refatoração

Não realizar imediatamente uma migração total e irreversível para uma arquitetura exclusivamente orientada a eventos.

A sincronização completa da base continua sendo uma regra de negócio.

Portanto, a implementação inicial deve preservar:

- sincronização completa dos códigos;
- botão manual de sincronização;
- um único run ativo;
- progresso global;
- capacidade de parar;
- capacidade de continuar ou recuperar;
- registros de erro;
- compatibilidade com o frontend atual sempre que possível.

A arquitetura futura poderá utilizar filas duráveis e eventos incrementais, porém a primeira entrega deve ser uma evolução compatível com o sistema existente.

A estratégia desejada é híbrida:

Sincronização completa paralela e confiável
+
estrutura preparada para eventos incrementais no futuro

No futuro, o sistema poderá utilizar:

Atualização incremental por evento
+
sincronização completa periódica de reconciliação

Mas não elimine a sincronização completa neste momento.

5. Objetivos técnicos

A nova implementação deve garantir:

1. processamento paralelo real;
2. múltiplos workers executando simultaneamente;
3. concorrência HTTP controlada;
4. distribuição segura dos códigos;
5. prevenção de processamento duplicado;
6. progresso persistido no banco;
7. retomada após interrupção;
8. retry individual;

9. timeout individual;
10. tratamento de falhas parciais;
11. finalização correta;
12. stop seguro;
13. lock confiável;
14. heartbeat;
15. observabilidade;
16. métricas de desempenho;
17. compatibilidade com o frontend;
18. capacidade de escalar;
19. proteção do ERP e do banco;
20. meta de conclusão de 3.000 códigos em 2 a 3 minutos, quando a latência externa permitir.

6. Análise obrigatória antes de alterar código

Antes de implementar qualquer alteração, faça uma inspeção completa do código atual.

Localize e analise:

- `kpi-sync-daily`;
- `kpi.tsx`;
- `clientes.tsx`;
- funções auxiliares utilizadas pela sincronização;
- tabelas de runs;
- tabelas de snapshots;
- tabelas de erros;
- tabela ou view utilizada pelo KPI;
- tabela de empresas/clientes;
- funções RPC relacionadas;
- triggers relacionadas;
- locks;
- cron jobs;
- chamadas `fetch`;
- autochamadas HTTP;
- atualizações de progresso;
- consultas de status;
- fluxo de parada;
- fluxo de finalização;
- tratamento de timeout;
- tratamento de erro;
- mecanismos de retry;
- qualquer uso de `EdgeRuntime.waitUntil`;
- qualquer uso de promessas não aguardadas;
- qualquer processamento após a resposta HTTP;
- qualquer dependência de estado em memória.

Produza primeiro um diagnóstico contendo:

6.1 Arquivos encontrados

Liste todos os arquivos envolvidos.

6.2 Tabelas encontradas

Liste:

- nome da tabela;
- finalidade;
- colunas relevantes;
- índices;
- constraints;
- RLS;
- funções que escrevem nela.

6.3 Fluxo atual

Descreva passo a passo:

```
Clique no frontend
→ chamada da function
→ criação ou leitura do run
→ obtenção dos códigos
→ processamento
→ atualização
→ encadeamento
→ finalização
```

6.4 Causa exata do travamento

Não faça apenas suposições genéricas.

Identifique no código:

- qual função inicia o worker;
- qual função processa o lote;
- como o próximo chunk é chamado;
- se a chamada é aguardada;
- se a resposta é retornada antes;
- se a promessa pode ser interrompida;
- se o próximo worker depende da execução anterior continuar viva;
- se existe timeout;
- se existe erro silencioso;
- se o lock não é renovado;
- se o heartbeat para de atualizar;
- se o progresso só é atualizado ao final do lote;
- se um erro em um código derruba o lote;

- se existe `Promise.all` sem isolamento;
- se o body da requisição está sendo lido mais de uma vez;
- se o run pode ser concluído cedo;
- se o run pode permanecer eternamente como `running`.

6.5 Riscos de concorrência

Verifique se dois workers podem:

- processar o mesmo código;
- incrementar o progresso duas vezes;
- atualizar o mesmo item simultaneamente;
- liberar o mesmo lock;
- finalizar o mesmo run;
- sobrescrever o status;
- registrar duas alterações;
- recalcular o KPI duplicadamente.

Somente depois desse diagnóstico apresente o plano de implementação.

7. Arquitetura de execução desejada

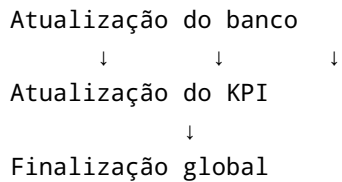
A arquitetura deve deixar de ser uma cadeia estritamente sequencial como:

```
Worker 1
→ chunk 1
→ chama worker 2
→ chunk 2
→ chama worker 3
```

Esse formato pode ser seguro para baixo volume, mas pode não atingir a meta de 2 a 3 minutos.

A arquitetura deve utilizar múltiplos workers paralelos:

```
Usuário clica em sincronizar
↓
Criação do run
↓
Persistência dos códigos
↓
Disparo de vários workers
↙   ↓   ↘
Worker 1 Worker 2 Worker 3 ... Worker N
↓     ↓     ↓
Cada worker reivindica códigos pendentes
↓     ↓     ↓
Processamento HTTP com concorrência controlada
↓     ↓     ↓
```



A responsabilidade de cada função deve ser separada.

Estrutura conceitual:

```
startDailySync
spawnInitialWorkers
executeWorker
claimNextBatch
processKpiChunk
processSingleCode
persistCodeResult
enqueueOrApplyKpiUpdate
recoverStaleWorkers
finalizeDailySync
stopDailySync
getDailySyncStatus
```

Os nomes podem ser adaptados ao código existente, mas as responsabilidades precisam estar claramente separadas.

8. Início da sincronização

A ação de início deve:

1. validar autenticação e autorização;
2. verificar se já existe run ativo;
3. impedir dois runs completos simultâneos;
4. adquirir o lock `kpi_daily`;
5. criar um novo registro de run;
6. capturar ou gerar uma lista estável dos códigos;
7. eliminar códigos duplicados;
8. registrar `total_codes`;
9. criar os itens de processamento;
10. registrar configuração inicial;
11. registrar horário de início;
12. definir o status como `running`;
13. disparar vários workers paralelos;
14. retornar imediatamente ao frontend.

A requisição inicial não deve permanecer aberta aguardando os 3.000 códigos.

Resposta esperada:


```
{
  "success": true,
  "run_id": "uuid",
  "status": "running",
  "total_codes": 3000,
  "worker_count": 10,
  "request_concurrency": 10
}
```

9. Persistência dos itens do run

Preferencialmente, cada código deve possuir um item persistido.

Utilize a estrutura existente se ela já suportar isso.

Caso não exista estrutura equivalente, proponha uma tabela semelhante a:

```
create table if not exists kpi_sync_run_items (
  id uuid primary key default gen_random_uuid(),
  run_id uuid not null,
  codigo text not null,
  status text not null default 'pending',
  attempts integer not null default 0,
  worker_id text,
  claimed_at timestamptz,
  heartbeat_at timestamptz,
  started_at timestamptz,
  finished_at timestamptz,
  next_retry_at timestamptz,
  last_error text,
  error_code text,
  previous_value jsonb,
  received_value jsonb,
  changed boolean not null default false,
  kpi_status text,
  kpi_error text,
  duration_ms integer,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);
```

Crie constraint única:

```
unique (run_id, codigo)
```

Estados sugeridos para o item:

```
pending
processing
retrying
completed
failed
stopped
```

Estados do KPI, se separado:

```
pending
processing
completed
failed
not_required
```

Não crie a tabela automaticamente sem antes verificar se as tabelas existentes podem ser reutilizadas ou estendidas.

10. Reivindicação atômica de códigos

Múltiplos workers não podem selecionar os mesmos códigos.

Implemente uma RPC transacional para reivindicar códigos.

Conceito:

```
select id, codigo
from kpi_sync_run_items
where run_id = p_run_id
and (
    status = 'pending'
or (
    status = 'retrying'
    and next_retry_at <= now()
)
or (
    status = 'processing'
    and heartbeat_at < now() - interval '2 minutes'
)
)
order by created_at
limit p_claim_size
for update skip locked;
```

Depois, dentro da mesma transação:

```
status = processing
worker_id = identificador do worker
claimed_at = now()
heartbeat_at = now()
attempts = attempts + 1
```

A RPC deve retornar somente os itens efetivamente reivindicados.

O uso de `FOR UPDATE SKIP LOCKED` ou mecanismo transacional equivalente é obrigatório para evitar que dois workers peguem o mesmo código.

Não faça:

```
SELECT pending
depois
UPDATE processing
```

em duas operações independentes, pois isso cria condição de corrida.

11. Estratégia de paralelismo

A configuração inicial desejada deve ser aproximadamente:

```
const WORKER_COUNT = 10;
const CLAIM_SIZE = 30;
const REQUEST_CONCURRENCY = 10;
const MAX_ATTEMPTS = 3;
```

Isso representa:

```
10 workers paralelos
x
10 requisições simultâneas por worker
=
até 100 chamadas HTTP simultâneas
```

Esses valores devem ser configuráveis por variável de ambiente ou configuração de sistema.

Exemplo:

```
KPI_SYNC_WORKER_COUNT
KPI_SYNC_CLAIM_SIZE
KPI_SYNC_REQUEST_CONCURRENCY
KPI_SYNC_MAX_ATTEMPTS
```

```
KPI_SYNC_REQUEST_TIMEOUT_MS  
KPI_SYNC_STALE_WORKER_SECONDS
```

Defina valores padrão seguros.

Não fixe valores de maneira que exija alteração de código para fazer tuning.

12. Meta de throughput

Para 3.000 códigos:

```
3 minutos = aproximadamente 16,7 códigos por segundo  
2 minutos = 25 códigos por segundo
```

A concorrência necessária depende da latência real da API.

Utilize a relação:

```
concorrência necessária  
≈ throughput desejado  
× latência da API  
× margem de segurança
```

Exemplo:

Se a chamada demora 3 segundos:

```
17 req/s × 3 s × 1,3 ≈ 66 chamadas simultâneas
```

Para 2 minutos:

```
25 req/s × 3 s × 1,3 ≈ 98 chamadas simultâneas
```

Portanto, 100 chamadas simultâneas é um ponto inicial coerente.

No entanto, a implementação deve medir:

- latência média;
- p50;
- p95;
- p99;
- taxa de erro;
- quantidade de timeouts;
- quantidade de retries;

- throughput;
- duração total.

Não declare que o SLA é garantido sem medir a API externa.

A aplicação deve ser construída para atingir a meta quando a latência e a capacidade do ERP permitirem.

13. Processamento interno do worker

Cada worker deve:

1. validar o `run_id`;
2. verificar o status do run;
3. verificar `cancel_requested`;
4. renovar heartbeat;
5. renovar lock;
6. reivindicar um lote;
7. processar o lote com concorrência controlada;
8. persistir o resultado de cada item;
9. atualizar heartbeat;
10. buscar mais trabalho enquanto houver tempo seguro;
11. encerrar antes do limite da Edge Function;
12. opcionalmente disparar um substituto;
13. tentar finalizar o run;
14. retornar um resumo.

Um worker não deve assumir que precisa processar todos os itens.

Ele deve processar somente enquanto estiver dentro de uma janela segura de execução.

Exemplo:

```
const SAFE_WORKER_EXECUTION_MS = 40_000;
const workerStartedAt = Date.now();

while (Date.now() - workerStartedAt < SAFE_WORKER_EXECUTION_MS) {
  const items = await claimNextBatch();

  if (items.length === 0) {
    break;
  }

  await processWithConcurrency(items);
}
```

Adapte o limite aos limites reais da Edge Function e da infraestrutura atual.

14. Concorrência HTTP controlada

Não use:

```
await Promise.all(
  allCodes.map(processCode)
);
```

Também não use um `Promise.all` com milhares de códigos.

Implemente concorrência limitada.

Pode utilizar uma função própria ou biblioteca compatível.

Exemplo conceitual:

```
async function processWithConcurrency<T>(  
  items: T[],  
  concurrency: number,  
  handler: (item: T) => Promise<void>,  
) : Promise<void> {  
  let cursor = 0;  
  
  async function workerLoop(): Promise<void> {  
    while (true) {  
      const currentIndex = cursor++;  
  
      if (currentIndex >= items.length) {  
        return;  
      }  
  
      await handler(items[currentIndex]);  
    }  
  }  
  
  const workers = Array.from(  
    { length: Math.min(concurrency, items.length) },  
    () => workerLoop(),  
  );  
  
  await Promise.allSettled(workers);  
}
```

Ou utilize `p-limit` caso seja compatível com o runtime.

Cada código deve ser isolado.

Uma falha em um código não pode cancelar os demais.

15. Timeout por requisição

Toda chamada ao ERP deve possuir timeout.

Utilize `AbortController`.

Exemplo:

```
async function fetchWithTimeout(
  url: string,
  timeoutMs: number,
): Promise<Response> {
  const controller = new AbortController();
  const timeout = setTimeout(
    () => controller.abort(),
    timeoutMs,
  );

  try {
    return await fetch(url, {
      method: "GET",
      signal: controller.signal,
      headers: {
        Accept: "application/json",
      },
    });
  } finally {
    clearTimeout(timeout);
  }
}
```

O timeout deve ser configurável.

Sugestão inicial:

15 a 30 segundos

Não permita que uma única chamada travada bloqueie indefinidamente um worker.

16. Retry individual

Implemente retry por código, não por lote inteiro.

Erros potencialmente transitórios:

```
408
425
429
500
502
503
504
timeout
falha temporária de rede
```

Erros permanentes ou não recuperáveis podem incluir:

```
código inválido
registro inexistente
resposta estruturalmente inválida
erro de autenticação permanente
regra de negócio rejeitada
```

Utilize backoff exponencial com jitter.

Exemplo conceitual:

```
Tentativa 1: imediata
Tentativa 2: 5 a 15 segundos
Tentativa 3: 20 a 45 segundos
Tentativa 4, se autorizada: 1 a 3 minutos
```

Para um run com meta de 2 a 3 minutos, evite que retries longos impeçam a finalização global.

Defina claramente:

- quantidade máxima de tentativas;
- erros repetíveis;
- erros definitivos;
- estado `retrying`;
- campo `next_retry_at`.

Ao atingir o limite:

```
status = failed
last_error = erro final
finished_at = now()
```


17. Idempotência

Toda operação deve poder ser repetida sem gerar efeitos duplicados.

A chave lógica mínima é:

```
run_id + codigo
```

Também considere uma chave de versão ou hash dos dados recebidos:

```
codigo + hash(payload_relevante)
```

Requisitos:

- o mesmo código não deve incrementar `processed_codes` duas vezes;
- a mesma alteração não deve incrementar `changed_codes` duas vezes;
- o KPI não deve ser recalculado duas vezes devido ao mesmo evento, salvo se a operação for idempotente;
- uma repetição após timeout deve resultar no mesmo estado final;
- o worker deve consultar o estado persistido antes de aplicar efeitos adicionais;
- a atualização da empresa deve comparar o valor atual com o valor recebido;
- somente mudanças reais devem contar em `changed_codes`.

Utilize:

```
is distinct from
```

para comparar valores com segurança em relação a `NULL`.

Exemplo:

```
update empresas
set
  associado_titular = p_associado_titular,
  erp_synced_at = now(),
  updated_at = now()
where codigo = p_codigo
and associado_titular is distinct from p_associado_titular;
```

O resultado do `UPDATE` deve informar se houve alteração.

18. Atualização transacional

Sempre que possível, a atualização do registro principal e o registro do resultado da sincronização devem ocorrer na mesma transação.

Crie uma RPC semelhante a:

```
apply_kpi_sync_code_result
```

Responsabilidades:

1. validar se o item pertence ao run;
2. validar se está em `processing`;
3. validar o `worker_id`, se necessário;
4. comparar valor anterior;
5. atualizar a empresa;
6. determinar se houve mudança;
7. atualizar o item do run;
8. registrar duração;
9. atualizar os contadores do run de forma segura;
10. criar o trabalho do KPI, caso necessário;
11. confirmar tudo atomicamente.

Evite a sequência:

```
UPDATE empresa
depois
UPDATE item
depois
UPDATE run
depois
INSERT evento
```

em chamadas independentes.

Se uma etapa falhar, o sistema pode ficar inconsistente.

Prefira uma única função transacional.

19. Estratégia de atualização do KPI

O KPI pode ser atualizado de duas formas, dependendo da regra atual.

19.1 Caso o KPI precise recalcular todos os códigos

O pipeline deve permitir que o KPI processe os códigos em paralelo.

Exemplo:

```
ERP conclui código
→ item de KPI fica disponível
→ workers de KPI processam em paralelo
```

Configuração inicial:

```
5 workers do KPI
×
10 processamentos simultâneos
=
50 operações simultâneas
```

19.2 Caso o KPI precise recalcular somente códigos alterados

O sistema pode:

1. consultar todos os códigos no ERP;
2. comparar os dados;
3. identificar os registros alterados;
4. enviar somente esses registros para atualização do KPI.

Exemplo:

```
3.000 códigos consultados
421 códigos alterados
421 atualizações do KPI
```

Essa opção preserva a sincronização completa porque todos os códigos foram verificados.

A diferença é que o segundo estágio é incremental.

19.3 Pipeline simultâneo

Não espere os 3.000 códigos do ERP terminarem para iniciar o KPI.

Faça pipeline:

```
ERP conclui código 1
→ KPI pode processar código 1

Enquanto isso:
ERP processa código 2, 3, 4...
```

A sincronização global só termina quando todas as etapas obrigatórias terminarem.

20. Regra de progresso

O progresso não pode depender somente de variáveis na memória da Edge Function.

O banco deve ser a fonte de verdade.

Calcule ou atualize de forma atômica:

```
total_codes  
pending_codes  
processing_codes  
retrying_codes  
processed_codes  
successful_codes  
failed_codes  
changed_codes  
kpi_pending_codes  
kpi_processing_codes  
kpi_completed_codes  
kpi_failed_codes
```

Defina claramente:

```
processed_codes =  
completed + failed
```

Ou outra definição consistente.

Não permita:

```
processed_codes > total_codes  
remaining_codes < 0
```

Sugestão:

```
remaining_codes =  
pending + processing + retrying
```

Não confie apenas em contadores incrementais se houver risco de duplicidade.

Quando necessário, reconcilie os contadores consultando os itens do run.

21. Finalização do run

O run não pode ser marcado como concluído somente porque um worker não encontrou itens naquele instante.

Pode existir outro worker com itens em processamento.

A finalização precisa ser transacional e baseada no estado global.

Condições para `completed`:

```
pending = 0
processing = 0
retrying = 0
failed = 0
kpi_pending = 0
kpi_processing = 0
kpi_failed = 0
cancel_requested = false
```

Condições para `completed_with_errors`:

```
pending = 0
processing = 0
retrying = 0
existe pelo menos um item failed
não existe falha estrutural
```

Condições para `failed`:

```
erro estrutural do pipeline
impossibilidade de continuar
falha de configuração
falha de autenticação global
lock irrecuperável
inconsistência crítica
```

Condições para `stopped`:

```
cancel_requested = true
nenhum novo processamento sendo iniciado
workers em andamento encerraram de forma segura
```

Ao finalizar:

- preencher `finished_at` ;
- liberar lock;
- calcular duração;
- registrar resumo;
- persistir métricas;
- impedir novos workers;
- permitir nova sincronização.

Utilize uma RPC de finalização com lock transacional para impedir que dois workers finalizem simultaneamente.

22. Stop seguro

O botão “Parar” não deve tentar cancelar fisicamente todas as requisições remotas de forma frágil.

O comportamento principal deve ser:

```
cancel_requested = true
```

Os workers devem verificar essa flag:

1. ao iniciar;
2. antes de reivindicar lote;
3. antes de cada grupo concorrente;
4. antes de retry;
5. antes de disparar novos workers;
6. antes de atualizar o KPI.

Ao detectar stop:

- não reivindicar novos códigos;
- permitir que requisições já iniciadas terminem ou sejam abortadas conforme política;
- marcar itens não iniciados como `stopped` ou mantê-los pendentes de acordo com a regra escolhida;
- aguardar que itens `processing` terminem ou expirem;
- atualizar o run para `stopped` ;
- liberar lock;
- preencher `finished_at` .

Documente claramente se o stop é:

```
graceful stop
```

ou:

```
hard stop
```

Prefira graceful stop.

23. Lock e heartbeat

O lock `kpi_daily` deve ser mantido.

Entretanto, ele precisa ser robusto.

O lock deve possuir:

```
lock_name  
run_id  
owner_id  
acquired_at  
heartbeat_at  
expires_at
```

Regras:

- somente o run ativo pode renovar o lock;
- heartbeat deve ser atualizado frequentemente;
- lock expirado pode ser recuperado;
- um run finalizado deve liberar o lock;
- um run interrompido deve liberar o lock;
- um run morto não pode bloquear novas execuções para sempre;
- a aquisição deve ser atômica.

O heartbeat do run e o heartbeat dos workers devem ser separados, se necessário.

24. Recuperação automática

A autochamada de worker não pode ser o único mecanismo de continuidade.

Implemente recuperação independente.

O sistema deve conseguir detectar:

```
run = running  
heartbeat antigo  
existem itens pending ou processing expirados  
não existem workers saudáveis
```

E então retomar.

Pode existir uma ação:

```
health
recover
continue
```

Ela deve:

1. localizar run ativo;
2. verificar heartbeat;
3. recuperar itens `processing` expirados;
4. movê-los para `retrying` ou `pending`;
5. renovar lock;
6. disparar workers substitutos;
7. registrar evento de recuperação.

Também pode ser configurado um cron periódico para verificar runs travados.

Entretanto, preserve compatibilidade com a implementação atual e não introduza cron desnecessariamente antes de analisar o ambiente.

25. Disparo dos workers

O primeiro `start` deve disparar vários workers.

Exemplo:

```
await Promise.allSettled(
  Array.from({ length: workerCount }, (_, index) =>
    invokeWorker({
      runId,
      workerId: `${runId}:${index}:${crypto.randomUUID()}`,
    }),
  ),
);
```

O disparo não pode depender da mesma requisição permanecer aberta processando os códigos.

Pode utilizar uma nova chamada HTTP para cada worker.

Se houver suporte confiável a processamento em background no runtime atual, avalie, mas não dependa exclusivamente de promessas não persistentes.

Cada worker pode, ao finalizar sua janela, disparar um substituto caso:

- o run ainda esteja ativo;
- existam itens pendentes;

- não exista cancelamento;
- o número de workers ativos esteja abaixo do desejado.

Evite explosão de workers.

Implemente controle de quantidade de workers ativos.

26. Controle de pressão

Mesmo que a API não declare limite, não assuma capacidade infinita.

O sistema deve responder a:

```
429
502
503
504
timeouts
aumento da latência
erros de conexão
saturação do banco
```

Implemente mecanismos como:

- limite de concorrência configurável;
- retry com backoff;
- jitter;
- circuit breaker simples, se necessário;
- redução temporária de concorrência;
- pausa entre lotes;
- métricas de erro.

Uma primeira versão pode usar concorrência fixa.

Entretanto, estruture o código para permitir concorrência adaptativa futuramente.

27. Concorrência adaptativa futura

Prepare a arquitetura para algo semelhante a:

```
Taxa de erro baixa
→ aumentar concorrência gradualmente

Muitos timeouts ou 5xx
→ reduzir concorrência
```

Resposta estabilizada
→ aumentar novamente

Não é obrigatório implementar algoritmo complexo na primeira entrega.

Mas evite espalhar constantes fixas em todo o código.

Centralize configurações.

28. Observabilidade

Registre logs estruturados.

Cada log deve incluir, quando aplicável:

```
run_id
worker_id
item_id
codigo
action
stage
attempt
duration_ms
http_status
error_type
timestamp
```

Exemplo:

```
{
  "level": "info",
  "event": "erp_code_processed",
  "run_id": "uuid",
  "worker_id": "worker-3",
  "codigo": "21344",
  "attempt": 1,
  "duration_ms": 2480,
  "changed": true
}
```

Evite logs genéricos como:

Erro ao processar

Prefira:

```
ERP_REQUEST_TIMEOUT
ERP_INVALID_RESPONSE
DATABASE_UPDATE_FAILED
KPI_UPDATE_FAILED
WORKER_CLAIM_FAILED
LOCK_RENEWAL_FAILED
RUN_FINALIZATION_FAILED
```

Não registre dados sensíveis desnecessariamente.

29. Métricas do run

Ao final, registre:

```
total_codes
successful_codes
failed_codes
changed_codes
unchanged_codes
retry_count
erp_duration_ms
kpi_duration_ms
total_duration_ms
average_request_ms
p50_request_ms
p95_request_ms
p99_request_ms
max_request_ms
effective_throughput
peak_worker_count
configured_concurrency
timeout_count
http_429_count
http_5xx_count
```

Se não for viável calcular percentis diretamente durante o run, armazene amostras ou durações individuais e calcule no final.

30. Frontend `kpi.tsx`

O frontend não deve controlar os códigos individualmente.

Ele deve:

- iniciar a sincronização;
- consultar o status;
- exibir progresso;

- permitir stop;
- exibir erros;
- atualizar os dados ao finalizar;
- não disparar chamadas por código;
- não gerenciar chunks;
- não manter estado operacional crítico somente no navegador.

Preserve polling inicialmente.

Não é necessário migrar para Realtime nesta etapa.

O polling deve:

- ter intervalo razoável;
- evitar chamadas sobrepostas;
- cancelar ao desmontar;
- tratar timeout;
- continuar após refresh;
- identificar run ativo;
- não criar novo run enquanto outro estiver ativo.

Exibir informações semelhantes a:

Sincronização em andamento

Total: 3.000

ERP processados: 2.350

ERP pendentes: 650

KPI atualizados: 1.900

Alterados: 421

Falhas: 2

Workers ativos: 10

Tempo decorrido: 01:42

Estimativa restante: 00:35

A estimativa pode ser calculada com base no throughput real.

Não exiba estimativa quando não houver amostra suficiente.

31. Frontend `clientes.tsx`

O módulo de clientes deve apenas consumir os dados persistidos.

Não deve:

- chamar o ERP por código;
- iniciar milhares de requisições;
- controlar workers;
- atualizar o KPI diretamente no navegador;

- possuir lógica de backend operacional.

Se o botão de sincronização estiver no módulo Empresa, ele deve somente chamar o endpoint de início.

Após conclusão:

- invalidar cache;
- recarregar os dados;
- atualizar filtros e paginação;
- refletir o novo `AssociadoTitular`.

32. Contrato da Edge Function

Preserve o contrato atual sempre que possível.

Caso existam ações via body:

```
{
  "action": "start"
}
```

```
{
  "action": "continue",
  "run_id": "uuid"
}
```

```
{
  "action": "stop",
  "run_id": "uuid"
}
```

Pode evoluir para:

```
start
worker
status
stop
recover
health
```

Evite quebrar chamadas existentes sem necessidade.

Caso seja necessário alterar o contrato:

1. documento;

2. atualize frontend e backend juntos;
3. mantenha compatibilidade temporária;
4. inclua validação de versão.

Não leia o body da requisição mais de uma vez.

Faça:

```
const body = await req.json();
```

uma única vez.

33. Segurança

Garanta que:

- somente usuários autorizados possam iniciar;
- somente usuários autorizados possam parar;
- o endpoint de worker não possa ser abusado externamente;
- workers validem um token interno ou segredo;
- o `run_id` seja validado;
- códigos não possam ser injetados livremente;
- service role seja usada somente no backend;
- nenhuma chave sensível seja enviada ao frontend;
- logs não exponham tokens;
- URLs do ERP sejam montadas com escaping adequado.

Avalie autenticação interna entre Edge Functions.

34. Tratamento da API GET externa

A API externa utiliza `GET`.

O sistema pode manter um endpoint interno de início usando `POST`.

Exemplo:

```
POST /kpi-sync-daily
body: { action: "start" }
```

Internamente:

```
GET API_DO_ERP?codigo=21344
```

Não exponha a semântica ruim da API externa ao restante da arquitetura.

35. Snapshot dos códigos

A sincronização precisa definir qual é o universo do run.

Evite que a lista mude de maneira inconsistente durante a execução.

Opções:

1. persistir todos os códigos no início;
2. persistir páginas conforme cursor, mantendo snapshot lógico;
3. utilizar timestamp/cursor estável.

Para aproximadamente 3.000 códigos, persistir os itens no início é aceitável e simplifica:

- total confiável;
- progresso confiável;
- idempotência;
- retry;
- stop;
- recuperação.

Verifique os snapshots atuais e reutilize-os se já cumprirem essa função.

36. Paginação e crescimento

Não faça uma única consulta que dependa de limites implícitos do Supabase.

Ao obter os códigos:

- pague;
- use cursor estável;
- ordene por campo determinístico;
- remova duplicados;
- valide nulos;
- normalize o código como string.

Se a tabela puder crescer muito, utilize:

```
updated_at + id
```

ou:

```
codigo + id
```

como cursor consistente.

37. Integridade dos códigos

Antes de inserir os itens do run:

- remover espaços;
- converter para string;
- validar formato;
- remover duplicados;
- ignorar código vazio;
- registrar códigos inválidos;
- não transformar códigos com zero à esquerda em número.

Utilize código como texto sempre que zero à esquerda for possível.

38. Atualização de AssociadoTitular

Mapeie cuidadosamente:

- formato retornado pelo ERP;
- tipo no banco;
- valores `true/false`;
- valores `0/1`;
- valores `"S"/"N"`;
- valores nulos;
- strings vazias;
- campos ausentes.

Crie uma função explícita de normalização.

Exemplo:

```
function normalizeAssociadoTitular(
  value: unknown,
): boolean | null {
  if (value === true || value === 1 || value === "1") {
    return true;
  }

  if (
    value === false ||
    value === 0 ||
    value === "0"
  ) {
    return false;
  }

  if (
    typeof value === "string" &&
    ["s", "sim", "true"].includes(value.trim().toLowerCase())
  ) {
    return true;
  }

  return null;
}
```



```

    ) {
        return true;
    }

    if (
        typeof value === "string" &&
        ["n", "nao", "não", "false"].includes(
            value.trim().toLowerCase(),
        )
    ) {
        return false;
    }

    return null;
}

```

Não use silenciosamente `Boolean(value)`, pois:

```
Boolean("false") === true
```

Registre respostas inválidas.

39. Transação de propagação para o KPI

Se o KPI depender diretamente de `AssociadoTitular`, avalie uma RPC que faça:

```

Atualizar empresa
+
Atualizar dados derivados do KPI
+
Atualizar item do run

```

na mesma transação.

Caso o recálculo do KPI seja pesado, use uma segunda etapa persistida.

Nesse caso, aplique o padrão de outbox:

```

Atualização da empresa
+
inserção do trabalho do KPI

```

na mesma transação.

Depois, workers do KPI consomem o trabalho.

Não permita que a empresa seja atualizada e o KPI seja perdido devido a falha entre operações.

40. Arquitetura futura com fila

A solução atual deve ser preparada para futura migração para:

Supabase Queues/PGMQ

Arquitetura futura:

erp-associado-sync
kpi-associado-sync
dead-letter

Mas não introduza essa migração total na primeira fase sem necessidade.

Organize o código para que o mecanismo de distribuição possa ser trocado.

Exemplo:

```
interface WorkDistributor {  
  claimBatch(...): Promise<SyncItem[]>;  
  acknowledge(...): Promise<void>;  
  retry(...): Promise<void>;  
  fail(...): Promise<void>;  
}
```

Hoje a implementação pode usar tabela e RPC.

No futuro pode usar PGMQ.

41. Arquitetura futura por eventos

No futuro, quando um dado mudar:

AssociadoTitular mudou
→ evento associado_titular_changed
→ KPI atualiza somente o código

A sincronização diária completa permanece como reconciliação.

Arquitetura final desejada:

Eventos incrementais
+
Reconciliação completa periódica

Isso oferece:

- atualização rápida;
- baixo custo;
- proteção contra eventos perdidos;
- consistência periódica;
- auditoria.

Não implemente essa substituição completa agora, mas deixe documentado o caminho de migração.

42. Plano de implementação em fases

Implemente preferencialmente em fases.

Fase 1 — Diagnóstico

- mapear código;
- mapear tabelas;
- identificar travamento;
- identificar riscos;
- documentar fluxo atual.

Fase 2 — Correção estrutural

- separar start e worker;
- persistir itens;
- implementar claim atômico;
- evitar execução longa;
- garantir heartbeat;
- garantir stop;
- corrigir finalização.

Fase 3 — Paralelismo

- múltiplos workers;
- concorrência interna;
- timeout;
- retry;
- métricas.

Fase 4 — Pipeline do KPI

- processamento paralelo;
- somente alterados, se compatível;

- outbox ou transação;
- finalização global.

Fase 5 — Frontend

- melhorar status;
- tempo decorrido;
- throughput;
- falhas;
- stop;
- recuperação após refresh.

Fase 6 — Testes de carga

- 100 códigos;
- 500 códigos;
- 1.000 códigos;
- 3.000 códigos;
- 5.000 códigos.

Fase 7 — Tuning

Testar:

```
5 workers × 10 = 50
10 workers × 10 = 100
10 workers × 15 = 150
```

Escolher o melhor valor com base em:

- tempo;
- erros;
- p95;
- estabilidade;
- carga do ERP;
- carga do banco.

43. Testes obrigatórios

Implemente testes unitários, integração e carga quando aplicável.

43.1 Testes unitários

- normalização de `AssociadoTitular`;
- classificação de erros;
- cálculo de backoff;
- cálculo de progresso;
- transição de status;

- validação de finalização;
- cálculo de estimativa;
- prevenção de duplicidade.

43.2 Testes de concorrência

Simular múltiplos workers.

Validar:

- nenhum código duplicado;
- nenhum contador duplicado;
- nenhum item perdido;
- nenhum run finalizado cedo;
- `SKIP LOCKED` funcionando;
- recuperação de worker morto.

43.3 Testes de retry

Simular:

- timeout;
- 429;
- 500;
- 503;
- resposta inválida;
- código inexistente.

43.4 Teste de stop

Durante 3.000 códigos:

- solicitar stop;
- impedir novos claims;
- encerrar com segurança;
- liberar lock;
- permitir novo run depois.

43.5 Teste de retomada

- matar worker;
- deixar item em `processing`;
- expirar heartbeat;
- rodar recover;
- reprocessar item;
- concluir run.

43.6 Teste de refresh

- iniciar sincronização;

- recarregar navegador;
- recuperar run ativo;
- continuar mostrando progresso;
- não criar novo run.

43.7 Teste de falha parcial

- 2.998 códigos com sucesso;
- 2 códigos com erro definitivo;
- status final `completed_with_errors`;
- relatório correto.

43.8 Teste de carga

Para 3.000 códigos, medir:

```
duração total
throughput
p50
p95
p99
timeouts
retries
falhas
uso do banco
uso da Edge Function
```

44. Critérios de aceite funcional

A implementação será aceita se:

1. o clique em sincronizar criar apenas um run;
2. o frontend receber resposta rapidamente;
3. múltiplos workers processarem simultaneamente;
4. nenhum código for processado duas vezes;
5. o progresso persistir no banco;
6. o refresh do navegador não perder o run;
7. o stop funcionar;
8. um worker travado puder ser recuperado;
9. falha de um código não derrubar o lote;
10. retries forem individuais;
11. o run não terminar cedo;
12. o lock for liberado corretamente;
13. o KPI refletir os dados atualizados;
14. erros forem auditáveis;
15. o sistema suportar pelo menos 3.000 códigos;
16. o processamento buscar conclusão em 2 a 3 minutos;
17. o frontend não disparar uma chamada por código;

- 18. o banco permanecer a fonte de verdade;
- 19. não houver contadores inconsistentes;
- 20. a implementação estiver documentada.

45. Critérios de aceite de desempenho

Executar teste real com 3.000 códigos.

Meta:

Tempo total preferencial: até 2 minutos
Tempo total máximo desejado: até 3 minutos

Registrar:

tempo médio da API
p95 da API
concorrência utilizada
taxa de erro
quantidade de retries
tempo do ERP
tempo do KPI
tempo total

Caso a meta não seja atingida, identificar claramente se o gargalo é:

- API do ERP;
- latência de rede;
- banco;
- função;
- atualização do KPI;
- contenção de lock;
- limite de conexões;
- configuração de concorrência.

Não mascarar o gargalo aumentando concorrência sem medição.

46. Critérios de aceite de confiabilidade

Em 10 execuções consecutivas com 3.000 códigos:

- nenhum run deve ficar eternamente `running` ;
- nenhum run deve exigir correção manual;
- nenhum código deve ser perdido;
- nenhum contador deve exceder o total;
- nenhum lock deve ficar permanentemente preso;
- nenhuma execução deve ser concluída cedo;

- falhas individuais devem ser registradas;
- a recuperação automática deve funcionar.

47. Entregáveis obrigatórios

Ao finalizar, entregue:

47.1 Diagnóstico

- fluxo atual;
- causa do travamento;
- riscos;
- gargalos;
- tabelas e arquivos envolvidos.

47.2 Plano aplicado

Explique:

- o que foi preservado;
- o que foi alterado;
- o que foi criado;
- o que ficou para fase futura.

47.3 Migrations SQL

Entregue migrations completas para:

- tabelas;
- colunas;
- índices;
- constraints;
- RPCs;
- triggers;
- permissões;
- RLS, quando aplicável.

47.4 Edge Functions

Entregue todos os arquivos alterados completos.

Não envie apenas trechos quando a alteração exigir contexto.

47.5 Frontend

Entregue alterações completas de:

- `kpi.tsx`;
- `clientes.tsx`;

- hooks;
- services;
- types;
- componentes de status;
- chamadas da function.

47.6 Configurações

Liste variáveis de ambiente necessárias.

47.7 Testes

Entregue:

- testes;
- comandos;
- cenários;
- resultados esperados.

47.8 Documentação operacional

Explique:

- como iniciar;
- como parar;
- como recuperar;
- como identificar run travado;
- como analisar erros;
- como ajustar concorrência;
- como testar 3.000 códigos;
- como fazer rollback.

48. Regras de implementação

Não faça uma reescrita cega.

Não remova estruturas existentes antes de entender sua finalidade.

Não crie tabelas redundantes sem necessidade.

Não quebre o frontend sem justificar.

Não faça chamadas massivas pelo navegador.

Não utilize concorrência ilimitada.

Não ignore timeout.

Não ignore idempotência.

Não dependa somente de promessas em memória.

Não marque sucesso enquanto houver item pendente.

Não esconda falhas.

Não use apenas logs como fonte de progresso.

Não permita que dois workers processem o mesmo código.

Não faça retry do lote inteiro por causa de um único código.

Não mantenha uma única requisição aberta durante toda a sincronização.

Não faça alteração de schema sem migration.

Não implemente somente uma solução teórica.

A entrega deve conter código funcional.

49. Qualidade do código

O código deve possuir:

- TypeScript estrito;
- tipos claros;
- funções pequenas;
- responsabilidades separadas;
- tratamento explícito de erros;
- validação de entrada;
- nomes descritivos;
- comentários somente onde agregarem valor;
- logs estruturados;
- ausência de duplicação;
- constantes centralizadas;
- contratos documentados.

Crie tipos semelhantes a:

```
type SyncRunStatus =  
  | "pending"  
  | "running"  
  | "completed"  
  | "completed_with_errors"  
  | "failed"  
  | "stopped";  
  
type SyncItemStatus =  
  | "pending"
```

```
| "processing"  
| "retrying"  
| "completed"  
| "failed"  
| "stopped";
```

Evite uso indiscriminado de `any`.

50. Exemplo de resultado esperado

Ao iniciar:

```
{  
  "run_id": "uuid",  
  "status": "running",  
  "total_codes": 3000,  
  "processed_codes": 0,  
  "remaining_codes": 3000,  
  "failed_codes": 0,  
  "changed_codes": 0,  
  "worker_count": 10,  
  "request_concurrency": 10  
}
```

Durante:

```
{  
  "run_id": "uuid",  
  "status": "running",  
  "total_codes": 3000,  
  "processed_codes": 2350,  
  "remaining_codes": 650,  
  "failed_codes": 2,  
  "changed_codes": 421,  
  "active_workers": 10,  
  "average_request_ms": 2480,  
  "throughput_per_second": 19.4,  
  "elapsed_seconds": 121,  
  "estimated_remaining_seconds": 34  
}
```

Ao finalizar:

```
{  
  "run_id": "uuid",  
  "status": "completed_with_errors",  
}
```

```
"total_codes": 3000,  
"processed_codes": 3000,  
"successful_codes": 2998,  
"failed_codes": 2,  
"changed_codes": 421,  
"unchanged_codes": 2577,  
"duration_seconds": 137,  
"average_request_ms": 2510,  
"p95_request_ms": 3840,  
"retry_count": 14  
}
```

51. Decisão arquitetural final

A entrega imediata deve utilizar:

```
run persistido  
+  
itens persistidos  
+  
claim atômico  
+  
múltiplos workers paralelos  
+  
concorrência HTTP controlada  
+  
timeout  
+  
retry individual  
+  
heartbeat  
+  
lock  
+  
stop  
+  
recuperação  
+  
finalização global
```

Configuração inicial sugerida:

```
WORKER_COUNT = 10  
CLAIM_SIZE = 30  
REQUEST_CONCURRENCY = 10  
CONCORRÊNCIA GLOBAL APROXIMADA = 100
```

Essa configuração deve ser validada por testes.

A arquitetura futura deve evoluir para:

```
Eventos incrementais
+
fila durável
+
reconciliação completa periódica
```

Porém, a sincronização completa atual não deve ser removida.

52. Ordem da resposta

Antes de alterar código, responda obrigatoriamente nesta ordem:

1. diagnóstico do fluxo atual;
2. arquivos envolvidos;
3. tabelas e funções envolvidas;
4. causa raiz do travamento;
5. riscos encontrados;
6. arquitetura proposta;
7. plano de migração;
8. alterações de banco necessárias;
9. alterações de backend necessárias;
10. alterações de frontend necessárias;
11. estratégia de teste;
12. implementação.

Depois dessa análise, faça a implementação completa.

Não pare somente no diagnóstico.

Não entregue apenas recomendações.

Não solicite aprovação entre cada etapa, salvo se encontrar uma ambiguidade impossível de resolver pelo código existente.

Faça a melhor implementação possível preservando compatibilidade, segurança e capacidade de rollback.